

GoPredict: Machine Learning for Vehicle Sales Optimization

CMPT 2500 – Machine Learning Deployment & Software Development

Aquiles Escarra

Angela Lekivetz

Victoriia Biragova

Komaljeet Kaur

Introduction

GoPredict predicts how long a vehicle will remain on the market based on historical data from Go Auto dealerships in Edmonton and the surrounding area. The project demonstrates a full MLOps process, from early experimentation in Jupyter notebooks to a deployed, containerized application capable of handling real-time predictions. Our goal was to create a reliable regression model that supports better inventory and pricing decisions while emphasizing reproducibility, scalability, and sound development practices.

Our project began in *Work Integrated Learning I*, where we tested several regression models using features such as vehicle price, mileage, make, and model. Exploratory data analysis showed that MSRP was a strong predictor of time to sale, so we segmented our model into three separate Random Forest regressors by MSRP range. This segmented approach improved overall performance and provided more precise results for each market tier. The outcome of that work was a deployed Streamlit application that allowed users to enter vehicle details and receive predicted days-on-market values.

MLOps Journey

In *Machine Learning Deployment and Software Development*, our focus shifted from experimentation to production. We began by breaking the original Jupyter notebook into separate Python scripts for data preprocessing, model training, and API development, turning the workflow into a maintainable system that could be deployed and scaled. A structured folder system separated code, data, and configuration files, supporting reproducibility and collaboration.

We connected the project to DagsHub for version control and remote data management, which allowed us to store datasets, track commits, and share model versions. MLflow was used to log one version of our trained model, capturing its parameters and performance metrics. This experience helped us understand how both tools can be scaled in future iterations to support full experiment tracking, versioning, and collaboration. Configuration management was handled through environment files and Docker Compose, ensuring consistent builds and simpler updates to parameters.

Docker was used to containerize the Flask API and supporting services so the project could run reliably across environments. Prometheus and Grafana were integrated to monitor performance by scraping metrics such as latency, uptime, and request counts. Once the containers were configured correctly, Grafana dashboards provided a live view of the system's behavior and stability.

The final deployment ran on Google Cloud Run, which handled container orchestration, scaling, and revision management automatically. These combined steps represented a major shift from isolated experimentation to a production-ready MLOps workflow capable of reproducibility, monitoring, and continuous improvement.

Implementation Challenges

Refactoring the original notebook into modular scripts required careful planning. Early versions of the project had most of the code in a single file, making it difficult to debug or reuse. Separating the workflow into scripts for preprocessing, training, and deployment improved the structure but introduced new dependencies that needed to be managed consistently.

Managing data and model artifacts across environments was another learning curve. Ensuring that datasets, trained models, and configurations stayed consistent between local, Docker, and cloud environments required coordination and version tracking through DagsHub and MLflow.

The most technical challenges came from debugging Docker networking and deploying to Google Cloud Run. The containers didn't initially communicate properly, and Prometheus couldn't scrape metrics until the network configuration was corrected. During deployment, port mismatches and authentication errors prevented the API from connecting to stored model files. Iterating through these issues taught us how container networking, monitoring, and cloud orchestration all fit together within an MLOps pipeline.

Overall, the experience reinforced that MLOps is more about process, structure, and discipline than model tuning. The long-term payoff came from building reliable systems grounded in solid software engineering practices.

Transformation Narrative

Transforming GoPredict from an experimental model into a deployed system changed how we approached development. Early in the project, our focus was on model accuracy, but it shifted toward building a reliable, repeatable workflow. Breaking the code into modular scripts and containerizing the system helped us think more like software engineers, not just data scientists. Deploying on Google Cloud Run and integrating Prometheus and Grafana showed how machine learning, infrastructure, and monitoring come together in practice. The experience highlighted that successful MLOps depends on structure, collaboration, and reproducibility rather than model tuning alone.

Current Limitations

Although GoPredict now functions as a complete end-to-end system, it still has several limitations. The dataset is limited to Go Auto dealerships in the Edmonton area, which limits the model's ability to generalize to other markets. Additional data from different regions, vehicle types, or seasons would help strengthen model performance and reliability.

The Random Forest models also provide limited interpretability. While they perform well, it is difficult to explain exactly how each feature contributes to the prediction. Tools such as SHAP or LIME could be applied in the future to make the model's decisions more transparent and easier to communicate to business users.

From a deployment perspective, the system could be improved by adding stronger automation and scalability. The current version handles predictions in real time but only supports a single instance at a time. Cloud Run scales automatically, yet cold starts still introduce short delays. A more advanced setup using a continuous integration and delivery pipeline, along with improved monitoring and model retraining processes, would help make GoPredict more efficient and production-ready.

Deployment and Monitoring

The final version of GoPredict was packaged into Docker containers so that all services could run in a consistent environment. Docker Compose was used to manage the Flask API along with Prometheus and Grafana, which allowed the entire setup to be started and tested locally before moving to the cloud. Prometheus scrapes metrics from the running containers, while Grafana allows us to visualize these metrics in real time through customizable dashboards. This setup gives us insight into our system's performance and helps to confirm that containerization is stable and responsive. Once the system was stable, we deployed it to Google Cloud. Each time a new item was pushed, Cloud Run automatically creates a new revision, which would help us to simplify version control and make rollback easy if necessary.

Future Work

Future development of GoPredict will focus on strengthening automation, scalability, and collaboration across the MLOps pipeline. Expanding the use of DagsHub and MLflow will allow for full data and model version tracking, making it easier to compare experiments and maintain a clear record of changes over time.

Implementing continuous integration and delivery (CI/CD) through GitHub Actions or Google Cloud Build will automate testing and deployment, reducing manual setup and configuration errors. Integrating these tools with Cloud Run would streamline the release process and ensure that new versions can be deployed safely and consistently.

On the modeling side, the next steps include expanding the dataset with additional dealerships, vehicle types, and seasonal data to improve model generalization. The team also plans to automate retraining so that models can update as new sales data becomes available. Enhancing the monitoring system with additional metrics for response times, model accuracy, and API uptime will help maintain reliability as the system scales.